

Teoría de Algoritmos

(75.29) Curso Buchwald - Genender

Trabajo Práctico 1:

Los Algoritmos Greedy son juegos de niños

---

11 de octubre de 2026

Lucio Romero Mascaró
113091

Joel Isaac Fernandez Fox
104424

Valentín Raunic
114081

Índice

1. Resumen **3**

2. El Problema **4**

2.1. Introducción 4

2.2. Planteamiento 4

2.3. Consecuencias de la optimalidad 5

3. Teoría Greedy **6**

3.1. Regla Greedy buscada 6

3.2. Pensamiento Greedy 7

4. Diseño del Algoritmo **8**

4.1. Lógica 8

4.2. Código 8

5. Analizando el Algoritmo **9**

5.1. Complejidad 10

5.2. Demostración de la Optimalidad 10

5.2.1. Impacto de la Variabilidad en la Optimalidad 12

5.3. Experimentos 12

6. Conclusión **15**

Bibliografía **15**

7. Anexo: Correcciones de Reentrega **16**

7.1. Impacto de la variabilidad de los valores en el tiempo de ejecución 16

7.2. Ejemplos de Ejecución y Casos Borde 16

1. Resumen

Se presenta un resumen del Trabajo Práctico 1, detallando los temas a tratar para ofrecer una lectura fluida, clara y precisa que permita comprender el desarrollo en su totalidad.

En primer lugar, se va a hablar sobre el problema propuesto, cómo fue el planteo de la resolución y cómo se logró llegar a que ésta misma pueda ser óptima para cualquier caso posible.

Se detalla la teoría necesaria sobre los *algoritmos Greedy*, enfocándose en lo más básico de la teoría, entre ella la *Regla Greedy*, la sucesión de los óptimos locales, el estado actual y el óptimo global. Para ver estos aspectos aplicados, se incluye un breve ejemplo de la idea en general y su analogía a conceptos geopolíticos (como el deontologismo).

Por otro lado, se detalla el diseño del algoritmo implementado en *pseudocódigo*, explicando anteriormente la lógica, cómo fue buscada y cómo fue pensada esa resolución. Adjuntándose al final la implementación realizada.

Se realiza un *análisis* del algoritmo planteado, mediante el estudio de la complejidad y una demostración matemática para validar la optimalidad del algoritmo, de forma clara y con ejemplos propuestos. Se incorporan múltiples experimentos empíricos de forma tal que se pueda visualizar gráficamente las medidas de tiempo sobre cierta cantidad de casos.

Finalmente, se expone una conclusión sobre lo aprendido, la dificultad del problema, y la particularidad de estos algoritmos y su pensamiento.

2. El Problema

Cuando Mateo nació, Sophia estaba muy contenta. Finalmente tendría un hermano con quien jugar. Sophia tenía 3 años cuando Mateo nació. Ya desde muy chicos, ella jugaba mucho con su hermano.

Pasaron los años, y fueron cambiando los juegos. Cuando Mateo cumplió 4 años, el padre de ambos le explicó un juego a Sophia: Se dispone una fila de n monedas, de diferentes valores. En cada turno, un jugador debe elegir alguna moneda. Pero no puede elegir cualquiera: *sólo puede elegir o bien la primera de la fila, o bien la última.* Al elegirla, la remueve de la fila, y le toca luego al otro jugador, quien debe elegir otra moneda siguiendo la misma regla. Siguen agarrando monedas hasta que no quede ninguna. Quien gane será quien tenga el mayor valor acumulado (por sumatoria).

El problema es que Mateo es aún pequeño para entender cómo funciona esto, por lo que Sophia debe elegir las monedas por él. Digamos, Mateo está "jugando". Aquí surge otro problema: Sophia es muy competitiva. Será buena hermana, pero no se va a dejar ganar (consideremos que tiene 7 nada más). Todo lo contrario. En la primaria aprendió algunas cosas sobre algoritmos greedy, y lo va a aplicar.

Para transformar esta situación en un problema algorítmico concreto, el desafío es el siguiente: Hacer un análisis del problema, y proponer un algoritmo greedy que obtenga la solución óptima al problema planteado: Dados los **n** valores de todas las monedas, indicar qué monedas debe ir eligiendo Sophia para si misma y para Mateo, de tal forma que se asegure de ganar siempre. Considerar que Sophia *siempre comienza* (para sí misma).

2.1. Introducción

Tranquilamente, se puede observar que el objetivo es uno solo: que *Sophia gane*. A eso sumemosle que juega "para los dos lados", siempre por beneficio suyo y Mateo está mas de *observador*.

Por último, lo más importante es que la elección va a ser si o sí entre 2 monedas y el valor entre ellas. Se puede preguntar:

- ¿Cómo se podría hacer que eligiendo x moneda (una entre esas dos), no haya otra moneda próxima (mayor a x) que otro la elija en el turno siguiente?
- ¿Cómo es posible crear un algoritmo Greedy en el cual sea óptimo para cualquier caso de monedas? ¿Qué es un algoritmo Greedy y qué significa que sea óptimo?
- ¿Existe un contraejemplo para ese supuesto algoritmo "óptimo"?

Una vez propuesto eso, sigamos pensando para ahora empezar con el planteo.

2.2. Planteamiento

Hacer que gane siempre Sophia, y tenemos la suerte también de que Sophia es la que elige por Mateo. Entonces, cómo podemos aconsejar a Sophia de tal forma que si siempre hace lo que le hayamos dicho, gane siempre?

La primer solución que se tiene que ocurrir es que entre las dos monedas a elegir agarre siempre la mayor en valor. Eso va a garantizar que siempre termine con una suma mayor que Mateo. O no?

Pero, ¿Qué tiene que elegir Mateo? ¿Qué tiene que elegir *Sophia* por Mateo?

Para que Sophia gane, tiene que tener la mayor cantidad de valor acumulado comparado con Mateo, entonces si Sophia agarra por cada par siempre la mayor, Sophia va a agarrar *por Mateo* la menor. Siempre para cada par de monedas, Sophia agarra la mayor para ella y para Mateo va a agarrar la menor entre estas dos. De manera *avariciosa*.

¿Sophia gana siempre?

2.3. Consecuencias de la optimalidad

Si gana siempre, significa que:

- Las indicaciones propuestas fueron óptimas.
- Si pasamos las instrucciones a un algoritmo, ese mismo es óptimo.
- Si es posible demostrar un solo contraejemplo, significa que no era óptimo.
- Va a haber que demostrar matemáticamente que es óptimo y no gana siempre.
- Se va a hablar en otro momento sobre el caso borde de un posible empate entre Sophia y Mateo.

Las pruebas a realizar de todo esto y mucho más se va a realizar en la lectura que sigue. Se recomienda leer detenidamente cada paso, y si no se entiende, releerlo.

A eso sumarle el pensamiento Greedy, hasta acá se dijo sutilmente como es éste. A continuación, se explica exáctamente eso: qué es un **algoritmo Greedy**, por qué se lo llama así, y más que nada cómo se puede implementar a este problema propuesto.

3. Teoría Greedy

"La codicia...es buena. La codicia es correcta. La codicia funciona." - Michael Douglas (película Wall Street 1987). Codicia es igual a Greed.

Es difícil de explicar bien qué es esta terminología. Pero se está hablando de un razonamiento *sin* visión general, sin la visión del "Big Picture" del problema o planteamiento.

Se asocia a la *avaricia* de la razón por la acción tomada. Y ahora piensen en Sophia, que quiere ganar siempre, que siempre agarra las monedas que a ella le convenga con tal de ganar en todo momento. Eso es avaricia.

Esa acción, ese pensamiento, esa *secuencia* de acciones se puede denominar como una acción Greedy. Siempre, en el momento en el que estoy y donde estoy, busco lo mejor para mí. Sin importar lo que venga después.

Ahora formalizandolo: se aplica una regla sencilla que nos permite obtener el/un **óptimo local** a mi **estado actual**. Aplicamos *iterativamente* esa regla, esperando que nos lleve al óptimo global (que sería la mejor solución encontrada de esa forma).

Un algoritmo es greedy cuando tenemos dicha regla sencilla que sólo mira el estado actual y que busca un óptimo local. Si no podemos enunciar cuál es el óptimo local, el algoritmo no es greedy o lo *estamos pensando mal*. Dicho de otra manera, el algoritmo construye una solución paso a paso mediante una secuencia *finita* de estados. En cada estado, el algoritmo selecciona la alternativa que parece ser la mejor en ese instante (la elección del óptimo local), sin reconsiderar decisiones pasadas ni pensar en las consecuencias de pasos futuros.

Pensar de esta manera, como veníamos diciendo sin la visión general del problema, *esperando* que nos lleve al óptimo global, los algoritmos Greedy nos pueden indicar varias cosas:

- No siempre dan el resultado óptimo.
- Demostrar que dan el resultado óptimo puede no ser fácil.
- Son intuitivos de pensar. Son fáciles de entender.
- Suelen funcionar rápido.
- Para problemas complejos pueden ser buenas aproximaciones.

Recordar la regla de: Sucesión de óptimos locales $\implies$ óptimo global. Esa misma nos va a permitir explicar el por qué un algoritmo es greedy. Por último, los algoritmos greedy poseen una propiedad de *subestructura óptima*: Una solución óptima al problema general contiene en su interior soluciones óptimas a los subproblemas derivados de la elección greedy.

3.1. Regla Greedy buscada

En el contexto del problema, el estado del juego queda determinado por la fila que queda de monedas y por el jugador de turno. Dado que Sophia juega su turno y además decide la jugada de Mateo, se propone la siguiente regla greedy: maximizar su propio beneficio inmediato y minimizar el asignado a su hermano. Esto lleva a **maximizar la distancia entre ellos dos**, en todo momento. Esa es la regla que se tiene que aplicar.

- **En el turno de Sophia:** Elegir la moneda de mayor valor entre los dos extremos disponibles.
- **En el turno de Mateo:** Sophia elige para él la moneda de menor valor entre los dos extremos disponibles.

Demostración de victoria:
Para ir pensando en la idea del por qué esta regla garantiza que Sophia nunca pierda, se puede analizar como avanza el juego en bloques de rondas formadas por dos turnos consecutivos (el de Sophia seguido por el de Mateo), cada dos turnos se eligen 2 monedas del total. Como Sophia gestiona las dos decisiones, su estrategia la ejecuta pensandolo como la regla greedy: maximiza su propia ganancia eligiendo el valor más alto de los dos extremos, y después, minimiza la elección de Mateo eligiendole el menor valor que quede.

Más adelante se va a detallar esta relación que asegura que en cada ronda, el valor que va a ir sumando Sophia va a ser mayor o igual al valor que recibe su hermano (terminando siempre mayor, ganandole a Mateo).

3.2. Pensamiento Greedy

Para entender mejor la naturaleza del pensamiento Greedy, vamos a analizarlo desde una perspectiva económica y filosófica.

Del lado económico, un algoritmo *Greedy* se comporta de manera muy similar al liberalismo clásico. Este sistema dice que si cada individuo busca maximizar de forma aislada su propio beneficio inmediato, la suma de egoísmos locales va a llevar de manera natural a un beneficio o bienestar (óptimo global) para *toda* la sociedad. En los algoritmos Greedy se aplica esto mismo: se espera que una sucesión de elecciones óptimas locales nos guíe hacia la solución óptima global.

Otra forma de plantearlo, desde el punto de vista filosófico, podemos contrastar este enfoque mediante el dilema del *Consecuencialismo (o Utilitarismo) contra el Deontologismo*. El consecuencialismo clásico busca maximizar el bienestar general pensando el impacto a largo plazo, asumiendo que a veces es necesario aceptar una pérdida momentánea o local para alcanzar un bien mayor en el futuro. El Deontologismo, va por el lado del cumplimiento estricto de una regla fija sin importar las consecuencias futuras (frecuentemente analizado a través de la tensión entre el egoísmo racional de Hobbes y la ética del deber de Kant).

A partir de esto, se puede decir que el pensamiento Greedy actúa como un Deontologismo. El algoritmo no es capaz de "empeorar" su estado local actual para beneficiarse en el futuro. En cada iteración piensa las consecuencias inmediatas y elige si o si en el momento presente la opción que maximiza su ganancia, omitiendo el impacto que tiene en futuras decisiones/iteraciones.

Teniendo en cuenta esta filosofía, pasemos ahora al diseño del algoritmo, manteniendo este enfoque Greedy en cada paso.

4. Diseño del Algoritmo

¿Por qué se elige para Sophia la mayor de las monedas y para Mateo la menor entre cada par? Si la regla hubiera omitido la maximización de la distancia, una posible regla greedy podría ser: "Para cada par, se elige la mayor", pero con eso mismo se le está dando la opción a Mateo de tener mayor puntaje. Y además el riesgo de Sophia, de que eligiendo la moneda de mayor valor en el momento, puede seguir después de esa una mayor a la elegida.

A veces la intuición falla, por eso hubo que pensar bien la técnica a realizar. Habiendo ya hablado de la secuencia de elecciones que se realizan para llegar a la solución del problema (Sophia gana), hay que pasar esa parte a la *lógica algorítmica*. ¿Cuál es el paso a paso en cualquier caso de monedas? ¿Cuál puede ser el peor caso posible? ¿Cómo interactúa el algoritmo con los datos del problema? ¿Cómo se pasan los datos del problema al idioma computacional?

4.1. Lógica

Una fila de n monedas, se puede implementar con un simple arreglo de números enteros. Donde cada posición representa una moneda, con su valor asociado. El turno va a empezar siendo de Sophia, y es donde va a empezar la elección.

La lógica se gestiona utilizando dos punteros que delimitan los extremos disponibles de la fila: `inicio` (inicializado en la primera posición) y `fin` (inicializado en la última). En cada turno, se evalúan los valores apuntados por los dos extremos. Después de cada elección, el puntero correspondiente se desplaza hacia el centro de la fila, reduciendo el espacio de búsqueda (porque ya seleccionamos dicha moneda).

Una vez que terminan las selecciones de una ronda, el control se alterna de forma automática. Siguiendo el marco teórico *greedy*, esta secuencia iterativa sigue hasta que el juego termina, escenario que ocurre cuando los punteros se cruzan, indicando que ya no quedan monedas disponibles en la mesa para seleccionar. Cada valor de la moneda seleccionada fue guardada en una variable `puntos` para ir guardando los resultados obtenidos.

4.2. Código

```
JuegoMonedas(monedas[1, 2, 3,..., n]):
    puntos_sophia = 0
    puntos_mateo = 0
    inicio = 0
    fin = n-1
    turno = True

    While inicio ≤ fin:
        If turno:
            // Turno de Sophia (maximiza su ganancia local)
            If monedas[inicio] > monedas[fin]:
                puntos_sophia = puntos_sophia + monedas[inicio]
                inicio = inicio + 1
            Else:
                puntos_sophia = puntos_sophia + monedas[fin]
                fin = fin - 1
```

```
17
18          Else:
19              // Turno de Mateo (Sophia minimiza la ganancia de el)
20              If monedas[inicio] ≥ monedas[fin]:
21                  puntos_mateo = puntos_mateo + monedas[fin]
22                  fin = fin - 1
23              Else:
24                  puntos_mateo = puntos_mateo + monedas[inicio]
25                  inicio = inicio + 1
26
27          turno = no turno // alternar el control del juego, indica el contrario del
    valor actual
28
29      Devuelvo (puntos_sophia, puntos_mateo)
```

5. Analizando el Algoritmo

Para analizar el algoritmo propuesto anteriormente, va a resultar útil plantear un ejemplo para analizar el comportamiento: Sea el arreglo de monedas $A = [1, 3, 4, 9, 6, 2]$.

- **Turno 1 (Sophia):** Evalúa los extremos $[1, 2]$. Selecciona 2. (Resto: $[1, 3, 4, 9, 6]$).
- **Turno 2 (Mateo):** Extremos $[1, 6]$. Sophia elige 1 para él. (Resto: $[3, 4, 9, 6]$).
- **Turno 3 (Sophia):** Extremos $[3, 6]$. Selecciona 6. (Resto: $[3, 4, 9]$).
- **Turno 4 (Mateo):** Extremos $[3, 9]$. Sophia elige 3 para él. (Resto: $[4, 9]$).
- **Turno 5 (Sophia):** Extremos $[4, 9]$. Selecciona 9. (Resto: $[4]$).
- **Turno 6 (Mateo):** Se le asigna la moneda restante 4.

Resultados: Puntos de Sophia $= 2 + 6 + 9 = 17$. Puntos de Mateo $= 1 + 3 + 4 = 8$. Sophia gana.

Se puede analizar el algoritmo implementado de varias formas, que vamos a estar explicando a continuación. Entre ellas, se va a hablar de la complejidad temporal y espacial algorítmica del algoritmo, y si es posible mejorarla.

También se habla de la optimalidad del algoritmo, anteriormente se hizo una breve presentación de ésta la cual ahora se va a explayar más y se va a demostrar matemáticamente por el método inductivo.

Por último se van a realizar experimentos sobre el mismo algoritmo, poniendolo sobre arreglos de distintos tamaños para hacer y visualizar mediciones de tiempo distintos.

5.1. Complejidad

Cuando se habla de *complejidad algorítmica*, buscamos en el algoritmo analizar el algoritmo de dos maneras: *espacial* y *temporal*. La complejidad espacial determina el espacio adicional ocupado en memoria por las estructuras de datos auxiliares, mientras que la complejidad temporal evalúa el tiempo de ejecución del algoritmo a medida que el tamaño de instancia (cantidad de monedas) crece. ¿Cómo se comporta el algoritmo?

Para este análisis se adopta el enfoque del *peor caso posible*[1], el cual se formaliza matemáticamente mediante la notación Big-Oh ($\mathcal{O}$).

La **complejidad temporal** del algoritmo diseñado para representar la sesión de juego de Sophia y Mateo es de $\mathcal{O}(n)$, donde n es la cantidad inicial de monedas en el arreglo. Esto debido a que se recorre todo el arreglo y por cada iteración lo único que se hace es comparar el elemento del principio con el del final, en base a eso y según de quién sea el turno, se elige el más alto o el más bajo. Por cada pasada simplemente se mueven los punteros hacia el centro (dependiendo de la moneda elegida).

- Comparar `monedas[inicio]` con `monedas[fin]` es $\mathcal{O}(1)$ (constante).
- Elegir el mayor o el menor según el turno es $\mathcal{O}(1)$.
- Mover los punteros de `inicio` y `fin` sumando o restando requiere tiempo $\mathcal{O}(1)$.

Como cada iteración del algoritmo saca una moneda y el ciclo se detiene cuando los punteros se cruzan, el bucle se ejecuta exactamente n veces. El tiempo de ejecución total se modela mediante la siguiente sumatoria de operaciones independientes:

$$\mathcal{T}(n) = \sum_{i=0}^{n-1} \mathcal{O}(1) = \mathcal{O}(n)$$

Por otro lado, el análisis de la **complejidad espacial** del algoritmo greedy implementado, se puede considerar un poco más directo. Si observan la estructura del código 4.2, la solución no duplica el arreglo original ni genera subarreglos intermedios mediante operaciones de copia. El algoritmo se limita a crear y actualizar un número fijo de variables (`inicio`, `fin`, `turno` y los contadores de puntaje). Acceder a los valores de un arreglo (representando las monedas), también tiene un valor constante. La complejidad espacial queda acotada de forma óptima en $\mathcal{O}(1)$.

5.2. Demostración de la Optimalidad

Que el algoritmo planteado sea *óptimo* implica que la estrategia *greedy* es suficiente para garantizar que Sophia gane siempre matemáticamente (obtener una sumatoria de valores mayor o igual a la de Mateo) en cualquier escenario posible, independientemente de la distribución de los valores. Se va a sacar de esta demostración el caso borde de un arreglo de tamaño n par donde todos los elementos son iguales, ya que va a resultar en un empate.

[1] Es importante destacar que en este algoritmo greedy, la distribución o el orden de los valores de las monedas no altera el analisis. Por lo tanto, el peor caso, el mejor caso y el caso promedio son iguales en términos numéricos de operaciones ejecutadas, lo cual simplifica la validación experimental.

Cuando se habla de algoritmos greedy, su demostración de que sean óptimos puede ser un tanto difícil. Una vez demostrada y habiendo justificado que sea óptimo, se puede asumir que el mismo algoritmo lleva una de las mejores soluciones de "manera" greedy.

Para demostrar que funcione siempre la regla greedy propuesta, se va utilizar la *Demostración por Inducción Matemática*. Se define nuestra variable de inducción k como la cantidad de *rondas* del juego. En cada ronda, se extraen exactamente 2 monedas (la elección de Sophia y la elección de Sophia sobre Mateo). Por lo tanto, un juego con $2k$ monedas equivale a k rondas.

Hipótesis Inductiva $P(k)$: Dada una fila de $2k$ monedas, la estrategia greedy garantiza que la ganancia acumulada de Sophia (S_{total}) será mayor a la ganancia acumulada de Mateo (M_{total}). Es decir, $S_{total} > M_{total}$ (gracias a haber descartado el empate).

Paso Base ($k = 1$):
El juego contiene $2(1) = 2$ monedas, guardadas en el arreglo $A = [c_1, c_2]$ (c de coin). Sophia, de manera avariciosa bajo su regla greedy, elegirá el máximo de los extremos. Su ganancia será $S_1 = \text{máx}(c_1, c_2)$. En consecuencia, el arreglo se reduce a una sola moneda, la cual Mateo está obligado a tomar. Su ganancia será $M_1 = \text{mín}(c_1, c_2)$. Al descartar el caso del empate (que esas mismas 2 monedas sean de igual valor), se cumple que:

$$\text{máx}(c_1, c_2) > \text{mín}(c_1, c_2) \implies S_1 > M_1$$

El caso base $P(1)$ es verdadero.

Paso Inductivo: ($P(k+1)$)
Se asume que $P(k)$ es verdadero. Eso significa que, para cualquier arreglo de $2k$ monedas, la estrategia garantiza que $S_{k_rondas} > M_{k_rondas}$. Ahora se tiene que demostrar que $P(k+1)$ también es verdadero para un arreglo inicial de $2(k+1) = 2k+2$ monedas.

En la primera ronda del juego con $2k+2$ monedas, los extremos disponibles son c_1 y c_{2k+2}. Sophia aplica la regla y elige $S_{ronda_1} = \text{máx}(c_1, c_{2k+2})$. Supongamos que elige c_1. El arreglo se reduce a los elementos desde c_2 hasta c_{2k+2}.

Después, Sophia ejecuta el turno de Mateo aplicando la regla greedy de maximizar la distancia para él: elegir el mínimo de los nuevos extremos disponibles (c_2 y c_{2k+2}). Mateo gana $M_{ronda_1} = \text{mín}(c_2, c_{2k+2})$.

Por la definición de la función mínimo[2], sabemos que $M_{ronda_1} \leq c_{2k+2}$. A su vez, como Sophia eligió c_1 por ser el máximo inicial, sabemos que $c_1 \geq c_{2k+2}$. Se puede deducir que:

$$S_{ronda_1} = c_1 \geq c_{2k+2} \geq \text{mín}(c_2, c_{2k+2}) = M_{ronda_1}$$

$$S_{ronda_1} \geq M_{ronda_1}$$

Luego de esta primera ronda, quedan exactamente $2k$ monedas en la mesa. Por nuestra **Hipótesis Inductiva**, se sabe que si ambos jugadores continúan aplicando la misma estrategia sobre estas $2k$ monedas restantes, se cumplirá que la ganancia de las rondas que siguen será $S_{resto} \geq M_{resto}$.

[2]La función mínimo, en el ámbito de matemáticas y programación, simplemente toma un grupo de números y devuelve el más pequeño de todos.

La ganancia total del juego será la suma de la primera ronda más el resto de las rondas:

$$S_{total} = S_{ronda_1} + S_{resto}$$

$$M_{total} = M_{ronda_1} + M_{resto}$$

Y como se ha demostrado que $S_{ronda_1} \geq M_{ronda_1}$, que no existe la opción de empate y asumido que $S_{resto} \geq M_{resto}$, al sumar ambas desigualdades se concluye que:

$$S_{total} > M_{total}$$

Por lo tanto, si $P(k)$ es verdadero, $P(k+1)$ también lo es. Queda demostrado por inducción que Sophia siempre va a obtener un puntaje mayor al de Mateo. En caso de que el tamaño inicial del arreglo n sea impar, después de $\lfloor N/2 \rfloor$ rondas completas, Sophia tendrá el turno final para tomar la moneda restante $c_{final} \geq 0$, lo que incrementará aún más su puntaje final, manteniendo la desigualdad $S_{total} > M_{total}$.

5.2.1. Impacto de la Variabilidad en la Optimalidad

Siguiendo el análisis del algoritmo, lo vamos a poner frente a una variabilidad grande entre los valores de las monedas. Por ejemplo, una distribución exponencial como $A = [1, 10, 100, 1000]$ con una varianza asimétrica[3], es indispensable destacar que la optimalidad no se ve afectada en ningún momento. La demostración por inducción desarrollada en la sección anterior no depende de la magnitud de los valores, sino de su relación de orden (mayor, menor o igual).

Dado que la regla greedy propuesta evalúa las desigualdades locales en cada turno (donde siempre se cumple que $c_1 \geq \text{mín}(c_2, c_{2k+2})$), y recordar que la estrategia greedy *no ve* las decisiones del impacto de valores futuros. Sin importar la diferencia de valor entre dos monedas es de valor 1 o de 1,000,000, Sophia siempre asegura matemáticamente la cota superior en la elección de esa ronda. Por lo tanto, la variabilidad del conjunto de datos *afecta la escala del puntaje final*, pero es imposible de invertir la condición de victoria $S_{total} > M_{total}$.

5.3. Experimentos

Se realizaron distintos experimentos en función de crear veinte arreglos de distintos tamaños, cada uno con una cantidad de elementos que van desde cien hasta diez millones de elementos equiespaciados, por lo que la separación entre cada número es de aproximadamente 526.310 números. Luego cada arreglo va a recibir números pseudoaleatorios que van desde el cero hasta cien mil (módulo `random`).

Como se observa en la Figura 1, la medida presenta una superposición prácticamente exacta con la recta de ajuste de Cuadrados Mínimos[4]. Esto evidencia un comportamiento lineal respecto al incremento de elementos en el arreglo, confirmando que la complejidad temporal es $\mathcal{O}(n)$.

[3] La varianza es una medida estadística que indica qué tan dispersos están los datos de un conjunto respecto a su valor promedio. El concepto de varianza asimétrica (o semivarianza) se utiliza cuando solo importa medir el riesgo o la dispersión en una dirección específica.

[4] La técnica de Cuadrados Mínimos es un método de optimización matemática que, en este análisis, permite encontrar la función lineal que mejor se ajusta al conjunto de datos experimentales, *minimizando* la suma de los

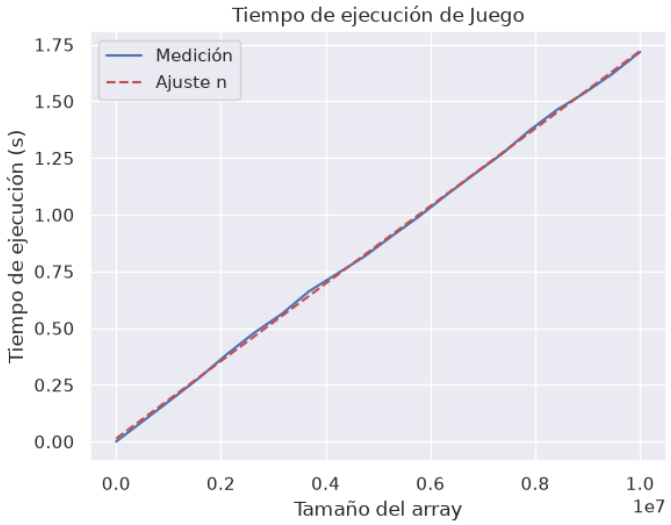


Figura 1: Tiempo de ejecución en función de la cantidad de elementos (n).



Figura 2: Error absoluto del ajuste por Cuadrados Mínimos.

En la Figura 2 se refuerza el argumento, observando que el error absoluto se mantiene acotado y en magnitudes despreciables a lo largo de todo el rango de n, validando la complejidad temporal del algoritmo *greedy* implementado.

errores al cuadrado entre los *tiempos de ejecución* medidos y la recta $\mathcal{O}(n)$.

6. Conclusión

El desarrollo de este trabajo práctico permitió analizar las capacidades y limitaciones del paradigma *Greedy* aplicado al problema. La implementación de una regla greedy asimétrica, asimétrica porque Sophia maximiza su beneficio local inmediato y minimiza el de Mateo. Demostró ser una solución óptima, logrando una complejidad temporal lineal $\mathcal{O}(n)$ y espacial constante $\mathcal{O}(1)$ que fue corroborada usando el ajuste de Cuadrados Mínimos.

Igualmente, el análisis más relevante de este escenario se basa en comprender las condiciones estrictas que habilitan la optimalidad del enfoque greedy. La estrategia implementada garantiza la victoria absoluta de Sophia únicamente por una restricción crucial de la consigna: ella tiene el control total sobre las decisiones de Mateo, anulando su capacidad de jugar a favor de él.

En conclusión, el problema propuesto los fundamentos teóricos en el diseño de algoritmos: las soluciones greedy son herramientas "poderosas", elegantes y rápidas, pero su éxito está vinculado a la estructura del entorno. Una pequeña modificación en las reglas del juego o en el comportamiento de los actores puede llegar a transformar un algoritmo lineal óptimo en una estrategia deficiente por el problema de no tener la visión del "Big Picture".

Se pudo demostrar que la estrategia greedy es un gran diseño y una gran solución, pero también un pensamiento peligroso al ignorar las fronteras del óptimo global.

Bibliografía

- Buchwald, Martín y Genender Ezequiel (2026). *Teoría de Algoritmos (75.29) - Apuntes de cátedra y clases*. Facultad de Ingeniería, Universidad de Buenos Aires.
- Hobbes, Thomas (1651). *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiastical and Civil*. London: Andrew Crooke.
- Kant, Immanuel (1785). *Fundamentación de la metafísica de las costumbres*. Riga: Johann Johann Christian Dieterich.
- Kleinberg, Jon y Éva Tardos (2005). *Algorithm Design*. Boston: Pearson Education.
- Smith, Adam (1776). *An Inquiry into the Nature and Causes of the Wealth of Nations*. London: W. Strahan y T. Cadell.

7. Anexo: Correcciones de Reentrega

En esta sección se va a tratar las correcciones solicitadas para la reentrega del Informe en cuestión. Logrando expandir el análisis de complejidad y añadiendo casos de ejecución particulares pudiendo visualizarlos graficamente.

7.1. Impacto de la variabilidad de los valores en el tiempo de ejecución

El algoritmo propuesto 4.2 tiene una complejidad temporal de $\mathcal{O}(n)$, es lógico analizar si la varianza en la magnitud de los valores de las monedas (ejemplo, tener monedas de valor 1 mezcladas con monedas de valor 1,000,000) genera impacto en el tiempo real de ejecución. ¿Lo genera?

La respuesta es que **no hay impacto algorítmico frente a la variabilidad**. La magnitud de los valores no altera la cantidad de pasos ni pone cuellos de botella en la CPU[5].

En cada iteración del `while` principal, el algoritmo sólo se fija en una única evaluación entre los dos extremos (`monedas[inicio] > monedas[fin]`) y una suma para el acumulador del puntaje. En las computadoras y lenguajes de programación de hoy en día, estas operaciones mantienen un costo temporal de $\Theta(1)$[6], manteniendo el comportamiento temporal tanto en el mejor caso, peor caso y caso promedio.

7.2. Ejemplos de Ejecución y Casos Borde

Para visualizar mejor el comportamiento del algoritmo, se realiza el seguimiento paso a paso de distintos casos borde que ponen a prueba el *algoritmo Greedy* planteado.

Caso 1: Todas las monedas de igual valor (Empate si o si)
Arreglo: $A = [5, 5, 5, 5]$

Turno	Jugador	Monedas	Extremos	Eligen	Ptos. Sophia	Ptos. Mateo
1	Sophia	$[5, 5, 5, 5]$	5 vs 5	5	5	0
2	Mateo	$[5, 5, 5]$	5 vs 5	5	5	5
3	Sophia	$[5, 5]$	5 vs 5	5	10	5
4	Mateo	$[5]$	5	5	10	10

Cuadro 1: Seguimiento de fila de monedas con valores iguales.

Resultado: Sophia = 10, Mateo = 10. Como hemos explicado, este es el único escenario matemático donde la estrategia queda en un empate inevitable.

Caso 2: Arreglos de tamaño mínimo (0 y 1 moneda)

- **Cero monedas ($A = []$):** El bucle principal `while inicio <= fin` no cumple la condición inicial. Los puntajes devuelven al instante 0.

[5] Un cuello de botella en la CPU pasa cuando el procesador se satura por un exceso de operaciones. En este caso, se descarta este problema ya que la variabilidad o el tamaño de los datos no afecta la *cantidad de pasos* del algoritmo. El esfuerzo que se le pide a la CPU se mantiene constante sin importar los valores de entrada.

[6] Cuando se habla de complejidad algorítmica, el símbolo de Θ representa la cota inferior y superior al mismo tiempo del algoritmo.

- **Una moneda ($A = [10]$):** Ambos punteros (`inicio` y `fin`) apuntan a la misma moneda. Sophia agarra la moneda (empieza ella). Se cruzan los punteros y el ciclo termina. *Resultado:* Sophia = 10, Mateo = 0.

Caso 3: Arreglo de picos alternados (test regla Greedy)

Arreglo: $A = [100, 1, 100, 1, 100]$

Este caso muestra cómo la regla Greedy (maximizar la ganancia propia y minimizar la del rival) se ejecuta bien sin caer en "trampas" numéricas:

Turno	Jugador	Monedas	Extremos	Eligen	Ptos. Sophia	Ptos. Mateo
1	Sophia	$[100, 1, 100, 1, 100]$	100 vs 100	100	100	0
2	Mateo	$[100, 1, 100, 1]$	100 vs 1	1	100	1
3	Sophia	$[100, 1, 100]$	100 vs 100	100	200	1
4	Mateo	$[100, 1]$	100 vs 1	1	200	2
5	Sophia	$[100]$	100	100	300	2

Cuadro 2: Seguimiento de fila de monedas con picos alternados.

Resultado: Sophia = 300, Mateo = 2. La optimalidad del algoritmo bajo la avaricia de controlar los dos turnos queda demostrado, maximizando la diferencia y que no afecte con arreglos "raros" buscando que se rompa la optimalidad.

Caso 4: Arreglo con el mejor valor en el medio (1, 1000, 1, 1, 1, 1)

Arreglo: $A = [1, 1000, 1, 1, 1, 1]$

Caso 4, con intención de que la moneda de mayor valor vaya a ser elegida para Mateo. Tratando de "romper" la optimalidad de la regla y estrategia ejecutada.

Turno	Jugador	Monedas	Extremos	Eligen	Ptos. Sophia	Ptos. Mateo
1	Sophia	$[1, 1000, 1, 1, 1]$	1 vs 1	1	1	0
2	Mateo	$[1, 1000, 1, 1]$	1 vs 1	1	1	1
3	Sophia	$[1, 1000, 1]$	1 vs 1	1	2	1
4	Mateo	$[1, 1000]$	1 vs 1000	1	2	2
5	Sophia	$[1000]$	1000	1000	1002	2

Cuadro 3: Seguimiento de fila de monedas con el valor ganador en el medio.

Resultado: Sophia = 1002, Mateo = 2. Pudiendo hacer que la elección entre el mejor valor (1000) o el peor (1) quede en el turno de Mateo, no logró absolutamente nada. La maximización y victoria segura de Sophia se mantuvo, una vez que vió que pudo minimizar por absoluto a Mateo y agarrar la moneda de valor 1000 para ella, lo ejecutó inmediatamente.

Caso 5: Arreglo con forma escalonada (1, 2, 3, ...)

Arreglo: $A = [1, 2, 3, 4, 5]$

Este caso tratamos de finalizar el análisis del algoritmo que no haya espacio de error con la regla Greedy, logrando la optimalidad por completo con cualquier caso de valores.

Turno	Jugador	Monedas	Extremos	Eligen	Ptos. Sophia	Ptos. Mateo
1	Sophia	$[1, 2, 3, 4, 5]$	1 vs 5	5	5	0
2	Mateo	$[1, 2, 3, 4]$	1 vs 4	1	5	1
3	Sophia	$[2, 3, 4]$	2 vs 4	4	9	1
4	Mateo	$[2, 3]$	2 vs 3	2	9	3
5	Sophia	$[3]$	3	3	12	3

Cuadro 4: Seguimiento de fila de monedas con valores escalonados.

Resultado: Sophia = 12, Mateo = 3. De nuevo, demostrada la optimalidad sin importar los valores de la fila de monedas. Y, teniendo cualquier otro arreglo M con cantidad par de monedas, cantidad impar, con valores en cualquier posición, Sophia y su avaricia de agarrar por Mateo y aumentar la distancia entre sus dos ganancias va a seguir manteniendo su victoria indiscutiblemente.